

Thread-per-Client Server

Das soll die grundsätzliche Architektur des Servers sein. Es muss gesagt sein, dass das nur der Grundriss dessen ist, wie so ein Server aufgebaut ist. Da es alles Real-Time sein soll - muss es Thread-Safe sein, da alle Threads gleichzeitig auf gewisse Elemente wie die Datenbank zugreifen können.

1. Server-Start

Dieses hat den Einstiegspunkt, die Main-Methode des Servers. Der Server wird auf einem Port gestartet, oder meldet, dass ein Fehler entstanden ist.

Der Server öffnet dann einen ServerSocket. Eine Endlosschleife ist dann das Herzstück, es läuft, bis ein Client sich verbindet.

Sobald das passiert, wird für diesen Client ein eigener ClientHandler bzw. eine Session erstellt und läuft dann in einem eigenen Thread. Der Server wartet dann wieder auf einen neuen Client.

2. Gemeinsames Protokoll

Dein Server nutzt ein textbasiertes Protokoll. Anstatt Bytes hin- und her zu schicken, nutzt er Strings, die durch Leerzeichen getrennt sind.

Es wird ein Paket definiert, welches da ist, um Information hin und her zu schicken. Dies ist das, was der Server dann schlussendlich nutzt für die Kommunikation.

Das Protokoll ist dann da, um aus rohem Text ein Packet zu generieren oder andersrum, also decoding und encoding von Packets. Wenn etwas Unerwartetes passiert oder Müll gesendet wird, gibt es hier eine Exception.

3. Session / ClientHandler

Da jeder Client einen eigenen Thread hat, läuft dieser in einer Run Methode. In dieser wird Input gelesen oder auch Output vom Server oder anderen Spieler hingeschickt. Das ganze soll Memory-Safe bzw. Ressourcen-Sicher mit try-with-resources gemacht werden, sodass In- und Output Streams sauber geschlossen werden, wenn es einen Fehler gibt.

Hier soll dann auch der Input über W/A/S/D an den Server geschickt werden oder Chat-Nachrichten an alle gesendet werden. Hier ist noch zu erwähnen, dass es einen Fixen

Tick geben muss, wie z.B. 24 mal oder 30 mal pro Sekunde um den Server nicht zu überlasten.

Auch muss sichergestellt werden, dass es beim Versenden der Pakete keine Probleme mit dem Multithreading gibt.

4. Zentrale In-Memory Datenbank

Die Registry soll als Datenbank dienen, hier soll der Spielstatus gespeichert sein, die Nicknames und Benutzer-IDs und auch alles andere, was wichtig ist. Auch hier muss es Threadsicher gecodet werden, da alle Clients darauf schreiben und zugreifen können.

Auch soll hierüber die Chatfunktion in einer separaten Klasse laufen, weil in den Datenbanken die Nicknames und alle ID's sind, kann mit Informationen von hier die entsprechenden Nachrichten dann an die Clients in den Chat gesendet werden.

5. Sketch

Diesen Sketch habe ich auf [Flowchart.io](https://flowchart.io) gezeichnet und dann von Nanobana (Auf Google Gemini 3.1) designen lassen (um im style der Präsentation zu sein). Hier ist in den Grundsätzen aufgezeichnet, wie die Kommunikation und die Aufgabenverteilung von Client und Server aufgeteilt ist.

